| | |
|---|---|
| 队伍编号 | MCB2102791 |
| 赛道 | B |

# 基于能量损失的 Transformer 和相似用户模型的推荐序列评估算法


## 摘 要

随着信息流和互联网的迅猛发展，网络越发成为人们获取信息的主要来源。为了有效的提升用户浏览信息的效率，准确推送用户喜爱的个性化的内容，成为当前的热门需求。其中推荐内容的排列顺序会影响用户的浏览体验。本文将对推荐序列的排序提出评估及用户反应预测的算法。

针对问题一和问题二，我们首先进行数据预处理，数据可视化及分析数据的基本情况，基于用户浏览时长，用户浏览时间，用户推荐内容，用户总点击量等特征，使用$k-means$算法对用户做聚类分析。我们提出我们采用 **Transformer 的深度神经网络模型**和**最相似用户数据的估计模型**相结合的**混合模型**以预测用户浏览各个物品的点击和浏览时间, 并把 Transformer 模型得到的结果标准化后求和得到**评估模型**。在完成数据预处理后，我们将用户 ID,推荐序列的内容，浏览时间，作为特征，浏览时长作为标签，基于用户消耗的精力（学习参数）和浏览时长的违反度设计损失函数。训练后，通过解码得到推荐序列的预测总点击量和总浏览时长。基于最相似用户数据的估计模型是通过计算目标用户与训练集用户的相似度，以选取训练集中与其最相似的用户，根据该特定用户在训练数据中的行为数据预测目标用户的行为。最终我们，将两个模型得到的结果加权求和便得到求解第二问的混合模型。问题二根据问题一训练得到网络模型就可以预测测试集中推荐序列的总点击量和总时长。

针对问题三，我们使用基于**集束搜索**和 **transformer** 的**强化学习**算法，基于总点击量和总浏览时长设计回报函数，从 N 条推荐内容选择 K 条推荐内容。结合集束搜索，这样根据强化学习的回报函数的选择可以得到最优的 K 条推荐内容，进而可以通过问题一提出的模型计算相应的总点击量和总浏览时长。



关键词：推荐算法，序列评估，神经网络，Transformer，集束搜索，强化学习

# 目录


一．问题重述 ..... 1
1.1 问题背景 ..... 1
1.2 需要解决的问题 ..... 1
二．问题分析 ..... 2
2.1 问题一分析 ..... 2
2.2 问题二分析 ..... 2
2.3 问题三分析 ..... 2
三．模型的假设 ..... 2
四．符号说明 ..... 3
五．模型的建立和求解 ..... 4
5.1 数据预处理 ..... 4
5.1.1 数据可视化及直观分析 ..... 4
5.1.2 数据分析 ..... 6
5.1.3 用户聚类 ..... 9
5.2 问题一求解 ..... 10
5.2.1 注意力机制和 Transformer ..... 10
5.2.2 基于 Transformer 的深度学习模型 ..... 12
5.2.3 基于最相似用户数据的估计模型 ..... 14
5.2.4 基于 Transformer 和相似用户估计的混合模型 ..... 16
5.3 问题二求解 ..... 16
5.4 问题三求解 ..... 17
5.4.1 集束搜索 ..... 17
5.4.2 强化学习 ..... 18
六．模型分析 ..... 21
6.1 优缺点分析 ..... 21
6.1.1 模型的优点 ..... 21
6.1.2 模型的缺点 ..... 21
七．参考文献 ..... 21
八．附录 ..... 22
8.1 txt2csv.ipynb ..... 22
8.2 analyze_data.ipynb ..... 29
8.3 create_dataset.ipynb ..... 35
8.4 classfy items by doc_info.ipynb ..... 52
8.5 split data by user id.ipynb ..... 57
8.6 fill test readily.ipynb ..... 60
8.7 create user df.ipynb ..... 66
8.8 test.ipynb ..... 72
8.9 evaluate.ipynb ..... 76
8.10 main.py ..... 78
8.11 utils.py ..... 79
8.12 transformer.py ..... 83

# 一．问题重述

## 1.1 问题背景

随着电商行业的迅猛发展，和互联网信息的大幅增加，目前人们已进入大数据时代，大数据时代的重要特征之一便是数据量巨大，信息超载已成为不得不面对的客观存在现象，对于想要定制满足自身兴趣偏好产品的客户来说，面对大量的产品配置信息，如何快速形成自己的个性化定制方案已是当务之急，而推荐系统的出现便高效地解决了这个难题。

现如今，互联网的处处都有推荐系统的身影。对用户来说，推荐系统就是解决信息过载情况下高效获取信息的问题。上网购物时，用户不可能在上亿量级的商品库中一一筛选，电商网站会根据历史的浏览、点击、购买行为为用户挑选满意的商品，提升了用户获取有用信息的效率；浏览新闻时，推荐系统也会根据用户的兴趣准备相应的新闻，如军事、娱乐、体育等；看各类短视频平台时，推荐系统更是占主导功能，推荐用户可能感兴趣的视频。可以说，推荐系统已经是互联网技术中不可缺失的一环，推荐系统也影响着我们生活的各个方面。对于平台来说，推荐系统就是主动地对用户的兴趣做预测，推送相应的内容，进而最大程度的吸引用户、留存用户、提高转化，从而达到公司商业目标的持续增长。不同平台的目标不同，如电商平台是通过用户的成交来赚取佣金，那么就会尽可能提高购买转化率（Conversion Rate，CVR），而短视频平台就会更加注重用户的留存时长等等。[1]

信息流作为推荐算法的主要应用场景，如何利用客户已有信息，给出适合用户的个性化推荐，包括考虑推荐商品的排序问题，是非常有意义的。

## 1.2 需要解决的问题

问题一是建立评估推荐序列总点击收益（序列中单条内容的点击量之和）和总时长收益（序列中单条内容的浏览时长之和）的数学模型，以及根据点击收益和时长收益对综合收益进行量化。需要着重考虑序列在评估模型设计中的影响，需要详细阐述如何考虑不同排列组合对收益大小的影响。

问题二是基于问题一设计的数学模型，预测测试集（test_data.txt）中推荐序列的总点击量和总时长（单位为秒），预测结果写入 result.csv 提交。文件包含三列：请求 ID、总点击量、总浏览时长。

问题三是假设有 N 条候选内容，从中选择长度为 K（N≥K）的最优推荐序列。在真实推荐场景中，推荐系统往往需要先采用计算复杂度更低的方式对序列集合进行剪

枝，圈定出少量候选序列进行精确收益评估。而剪枝策略的目标是保证候选序列集更大可能包含最优序列。论文将详细阐述建模思路，以及剪枝策略的精准度和时间复杂度。

# 二．问题分析

## 2.1 问题一分析

问题一需要构造一个评估推荐序列总点击收益（序列中单条内容的点击量之和）和总时长收益（序列中单条内容的浏览时长之和）的整合模型，我们采用 Transformer 的深度神经网络模型和最相似用户数据的估计模型相结合的混合模型。Transformer 的深度神经网络模型首先给定每个用户的能量函数Energy(t), 记录浏览内容带来的精力损失。将当前能量，用户浏览时长，用户观看时间，推荐内容类型作为encoder输入，输出为用户的精力损耗。由于内容是按照列表的序列顺序进行输入的，结合 Transformer 的注意力机制，是可以学习到内容顺序对用户浏览带来的影响。基于总点击量和实际点击量设计损失函数。网络结构采用引入注意力机制的深度神经网络。基于最相似用户数据的估计模型是通过计算目标用户与训练集用户的相似度，以选取训练集中与其最相似的用户，根据该特定用户在训练数据中的行为数据预测目标用户的行为。最终我们，将两个模型得到的结果加权求和便得到求解第二问的混合模型，同时根据基于 Transformer 的深度神经网络模型设计综合收益的评价模型。

## 2.2 问题二分析

根据问题一训练得到网络模型就可以预测测试集中推荐序列的总点击量和总时长。

## 2.3 问题三分析

问题三需要对用户的推荐内容序列进行剪枝，圈定出少量候选序列进行精确收益评估。我们采用基于集束搜索和 Transformer 的强化学习模型。强化学习模型可以基于设计的回报函数选出点击量高和浏览时间长的推荐序列，Transformer 模型可以很好的提取特征和分析序列信息。在强化学习的搜索采样中选择集束搜索将很大程度上提高搜索效率和提高解的质量。

# 三．模型的假设

1. 假设测试集中的用户爱好分布符合训练集中的用户爱好分布。
2. 假设每个用户的精力是一样的，但看完一项内容之后精力损耗是不一样的，看完物品后的精力损耗也是模型要训练学习的参数。
3. 假设用户在能量剩余小于等于 0 时不在继续浏览。

4. 假设相似的用户的被推荐内容是相似的。

## 四．符号说明

| 符号 | 说明 |
| --- | --- |
| $Energy(t)$ | 用户在 t 时刻的剩余能量（精力） |
| $D_t$ | 用户的浏览时长 |
| $P^{item}$ | 推荐内容序列 |
| $D_{ij}$ | 用户第 i 批数据的第 j 个内容的实际浏览时长 |
| $P_{ij}$ | 用户第 i 批数据的第 j 个内容的预测浏览时长 |
| $E$ | 用户初始能量 |
| $N^i_{Clickeds}$ | 测试集中第 i 条推荐内容用户的总点击量 |
| $N^i_{\mathrm{Durations}}$ | 测试集中第 i 条推荐内容用户的总浏览时长 |
| $NA^i_{Clickeds}$ | 基于深度神经网络模型预测的第 i 条推荐内容用户的总点击量 |
| $NB^i_{Clickeds}$ | 基于相似用户估计预测的第 i 条推荐内容用户的总点击量 |
| $NA^i_{\mathrm{Durations}}$ | 基于深度神经网络模型预测的第 i 条推荐内容用户总浏览时长 |
| $NB^i_{\mathrm{Durations}}$ | 基于相似用户估计预测的第 i 条推荐内容用户的总浏览时长 |
| $H^{enc}$ | 用户属性及用户请求发出的时间的嵌入向量 |
| $H^{\ell}$ | 第 l 层的嵌入向量 |
| $Q^{\ell}$ | 第 l 层的查询值 |
| $K^{\ell}$ | 第 l 层的键值 |
| $\mathrm{V}^{\ell}$ | 第 l 层的价值 |
| $p_t^{dec}$ | 第 t 个内容的嵌入向量 |

# 五．模型的建立和求解

## 5.1 数据预处理

### 5.1.1 数据可视化及直观分析

本小节将会对用户信息等出现的相关特征做可视化处理，针对用户数据做相关的聚类分析和用户画像。

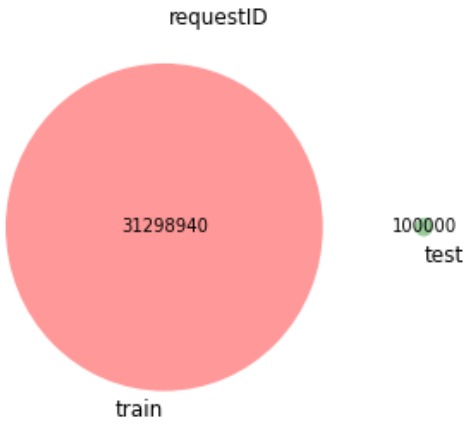



图 1：训练集和测试集的请求 ID

训练集数据有 31298940 条，测试集数据有 100000 条。数据量很大，首先我们要对训练数据和测试数据有一些基本认识。




图 2：训练集和测试集的用户 ID

我们发现训练集一共出现 1026531 个用户，测试集一共出现 71595 个用户。其中训练集和测试集有 68408 个相同的用户。占测试集用户总数的 95.55%。这说明分析总

结用户画像的重要性。

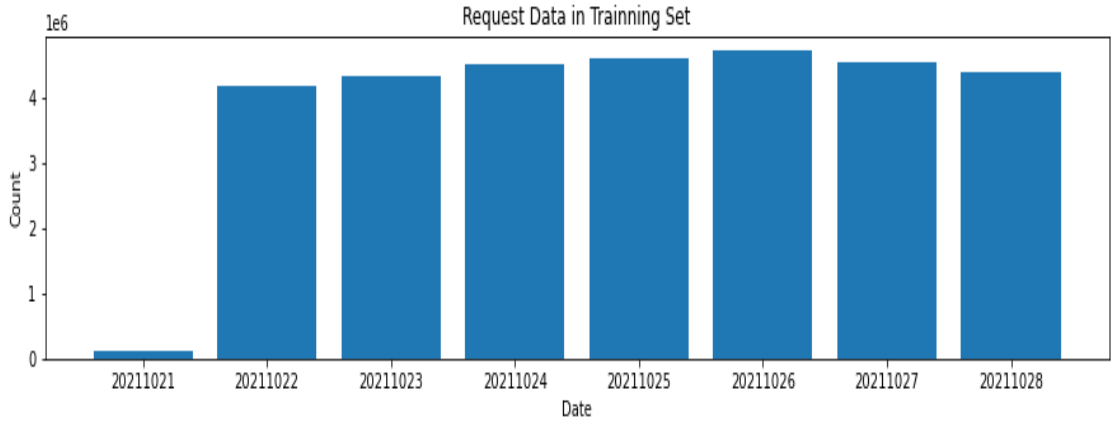



图 3：训练集中用户浏览时间统计（天）

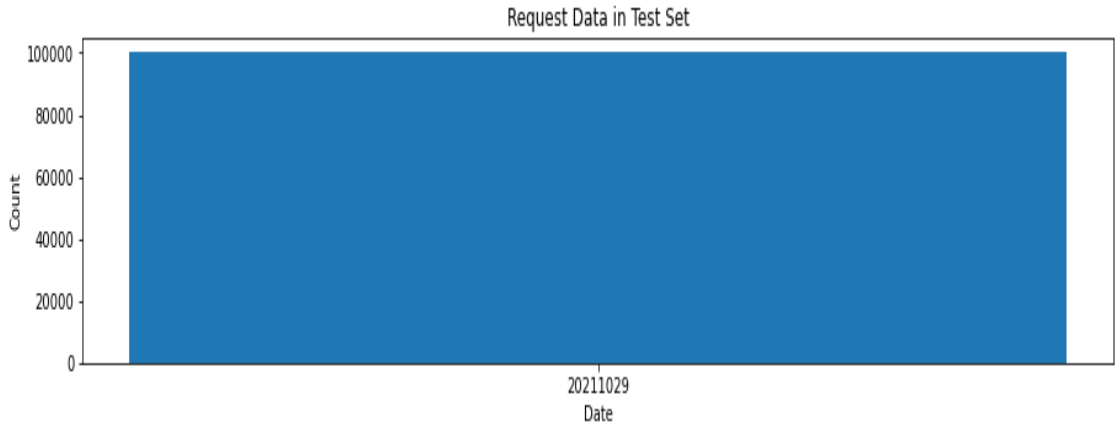



图 4：测试集中用户浏览时间统计（天）

数据的请求时间集中在 2021 年 10 月 22 日到 2021 年 10 月 28 日。每天的请求量基本相同，所以我们考虑以小时为单位，以天为周期。测试数据涉及到的时间是一天。所以只需要考虑一天内不同的时间点的影响即可。在后续的模型建立和分析中我们考虑用户在一天的不同时间段的浏览情况，作为分析的特征。

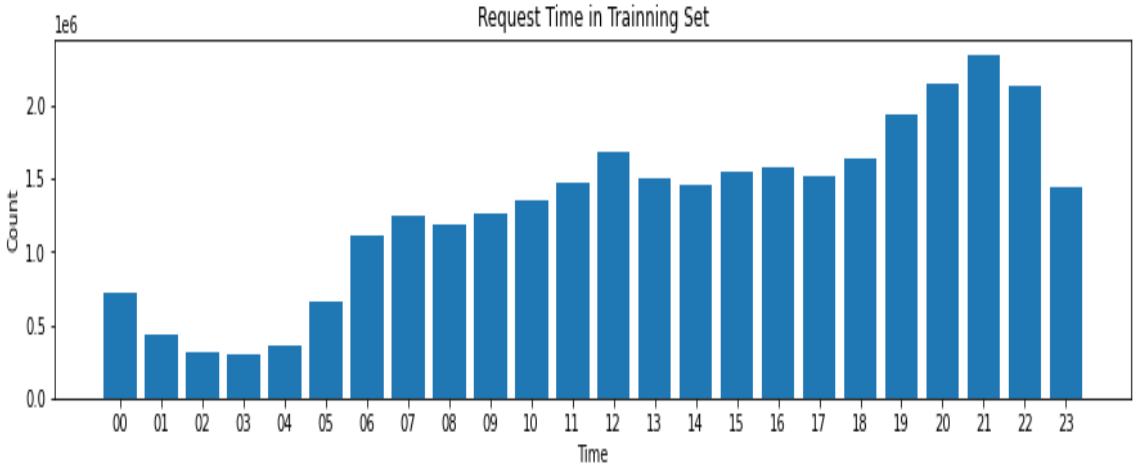



图 5：训练集中用户浏览时间分布（按小时）

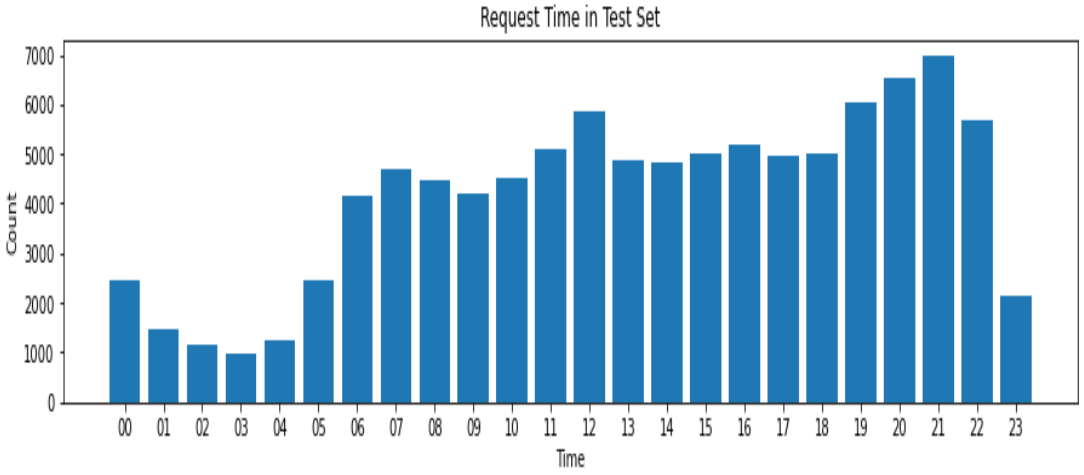



图 6：测试集中用户浏览时间分布（按小时）

测试集和训练集之间的请求时间的分布基本相同，均呈现凌晨时间较少，早上 6 点到晚上六点 6 点浏览数量较为稳定，中午会达到一个小峰值。19 点到 22 点是浏览的高峰期。训练集合测试集的分布大致相同也从侧面体现我们假设 1 的合理性。用户的浏览时间会作为后续的深度神经网络训练中的特征。

### 5.1.2 数据分析

按照内容类别对内容做分类如下表所示。发现有 34 个未知图文内容， 947 个未知视频内容， 将这些内容各归为一类. 最终将 2510703 个不同内容压缩为了 1485 个不同类别。

| index | contentID | contentType | contentCategory |
|---|---|---|---|
| 785196 | 133657464327 | video | A_0_24:0.653325,A_25_29:0.170250,A_30_39:0.134... |
| 535841 | 133658459211 | video | A_0_24:0.947994,A_25_29:0.003303,A_30_39:0.032... |
| 185162 | 505670245 | news | 两性/两性健康 |
| 186246 | 505857707 | news | 两性/两性健康 |
| 187525 | 506010721 | news | 两性/两性健康 |
| ... | ... | ... | ... |
| 2438682 | 133687111738 | video | NaN |
| 2439143 | 133687211179 | video | NaN |
| 2439454 | 133687293980 | video | NaN |
| 2443290 | 133688111043 | video | NaN |
| 2444802 | 42953250153 | video | NaN |

图 7：用户内容分类

训练集共有 31298940 条，数量过大，考虑将其按照发出请求的用户进行分类。对于 ID 为 2215314710 的用户的 30 条数据举例如下，

| index | userID | requestID | date | time | sequence |
|---|---|---|---|---|---|
| 1398624 | 2215314710 | 1107657355_1635127749510_1516 | 20211025 | 10 | 133675663351:1:0;133666769451:0:0;133657930533... |
| 1398625 | 2215314710 | 1107657355_1635128373562_8840 | 20211025 | 10 | 507316535:1:575;507422872:0:0;507330930:0:0;50... |
| 1398626 | 2215314710 | 1107657355_1635128522653_1783 | 20211025 | 10 | 133679055330:0:0;507430909:0:0;507349491:0:0;5... |
| 1398627 | 2215314710 | 1107657355_1635128626150_7090 | 20211025 | 10 | 507372660:0:0;133679039685:0:0;133678107347:0:... |
| 1398628 | 2215314710 | 1107657355_1635239789760_5314 | 20211026 | 17 | 133679271941:0:0;507293451:0:0;133679387259:0:... |
| ... | ... | ... | ... | ... | ... |
| 23307623 | 2215314710 | 1107657355_1635339623330_5162 | 20211027 | 21 | 133679286334:0:0;133672776393:1:1180;133672776... |
| 26439499 | 2215314710 | 1107657355_1634977820831_4298 | 20211023 | 16 | 508928441:0:0;133563467351:0:0;509021636:0:0;5... |
| 26439500 | 2215314710 | 1107657355_1635127684490_6841 | 20211025 | 10 | 133664140800:0:0;133676005033:0:0;506827267:0:... |
| 26439501 | 2215314710 | 1107657355_1635129329953_3958 | 20211025 | 10 | 133679736249:0:0;133666550165:0:0;133670640072... |
| 29569760 | 2215314710 | 1107657355_1634998692580_4466 | 20211023 | 22 | 133674063832:0:0;133664916157:0:0;506788060:0:... |

图 8：训练集推荐内容分类

对于上述 1485 中的类别，定义用户对每个类别所含所有内容的点击率为

$$Clicked = \frac{\text{该类别点击次数总和}}{\text{该类别被推荐次数总和}},$$

定义用户对每个类别所含所有内容的平均浏览时长为

$$Duration = \frac{\text{该类别浏览时长总和}}{\text{该类别点击次数总和}},$$

则可得到$Clickeds \in R^{1485}$和$Durations \in R^{1485}$.对于 ID 为 2215314710 的用户的 30 条数据举例如下，

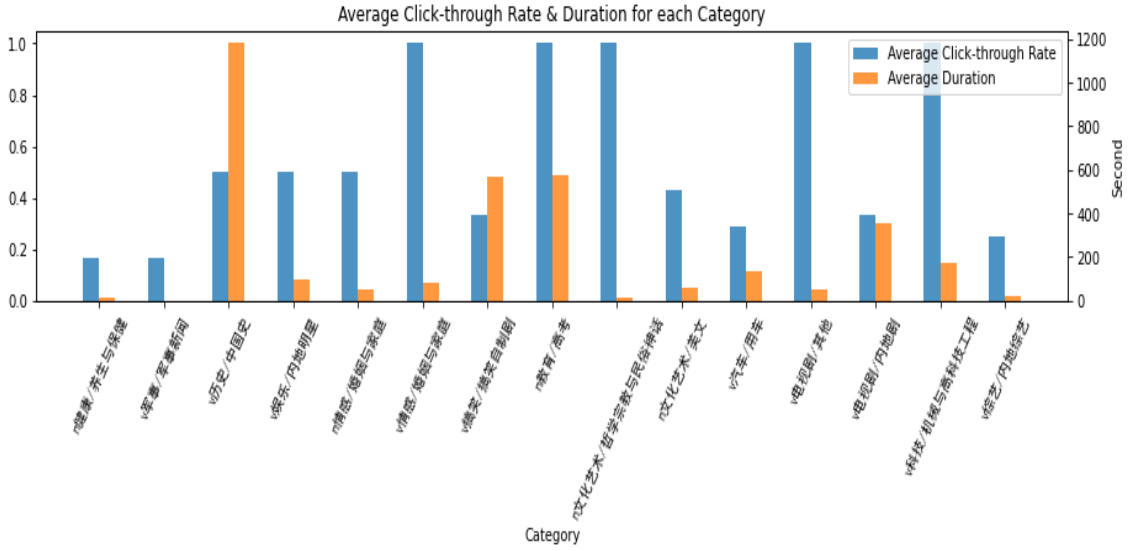



图 9：平均点击率和点击时长图

从该图中可以发现，纵观改用户产生的所有 30 条数据，其对情感，电视剧及科技的点击率几乎最高，均为百分之百；而点击后浏览时长最长的为中国史，平均浏览时长高达 20 分钟。

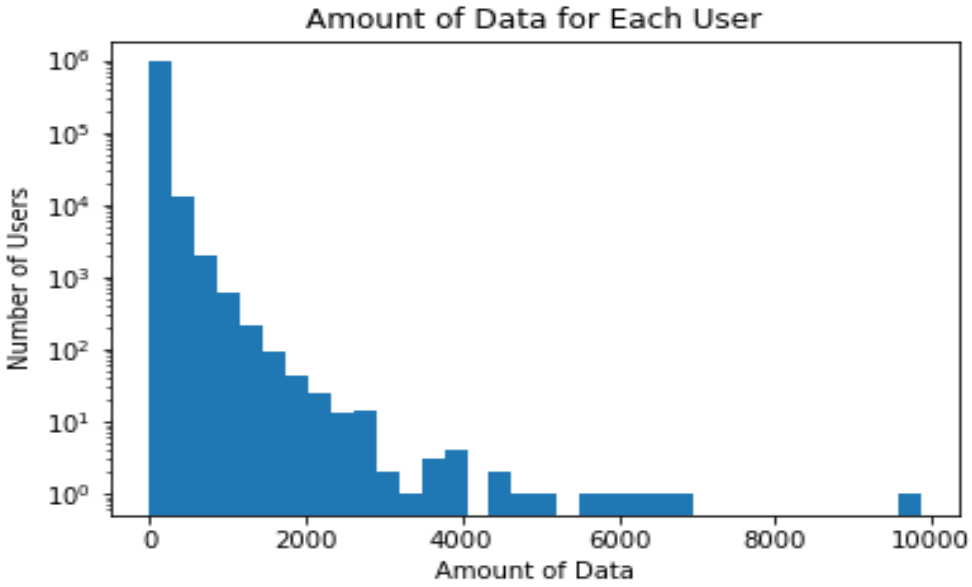



图:10：用户人均请求数

从中可以发现，大部分的用户只发出了少量的请求。具体来说，发出 1，2，3，4，5 个请求的用户数量分别为 234333，109116，63965，44344，33700 个．每个用户发出的请求平均数为 32.67，我们将请求数大于该值的用户定义为**忠实用户**，忠实用户共有 218744 个。

### 5.1.3 用户聚类

由于用户数量较大，我们对用户群体做分类，如果算力不满足的话可以用用户聚类后的结果作为用户画像。为了节省计算时间，我们考虑将具有相似浏览爱好的用户归到一个类别。根据用户浏览时长，用户浏览时间，用户推荐内容，用户总点击量等特征，使用$K-means$算法[2]做用户聚类分析。

$K-means$ 算法的工作机理是：把 $n$ 个样本点分为 $k$ 个簇，使得各簇内的样本点具有较高的相似性，而各簇间的样本点相似程度较低，相似度的计算是依据一个簇中样本点的平均值来进行。该算法根据聚类性能指标最小化原则，通常使用的聚类准则函数是簇内的各个样本点到该簇中心的误差平方和最小。

算法具体流程如下：

（1）在样本数据集 $D$ 中选择 $k$ 个样本点，将 $k$ 个样本点值分别赋给初始聚类中心（$\mu_1^{(1)},\mu_2^{(1)},...,\mu_k^{(1)}$）；

（2）第 $j$ 次迭代时，对样本点 $D$ 中的所有点 $P_t(t=1,...,n)$，依次计算到各簇中心 $\mu_i^{(j)}$ 的欧氏距离 $d(t,i)$，其中 $d(t,i)=\sqrt{(P_t-\mu_i^{(j)})^2}$；

（3）找出 $P_t$ 关于 $\mu_i^{(j)}$ 的最小距离，将 $P_t$ 归入到关于 $\mu_i^{(j)}$ 距离最小的簇中；

（4）更新各簇的聚类中心，$\mu_i^{(j+1)}=\frac{1}{n_i}\sum_{t=1}^{n_i}P_{it}$

（5）计算数据集 $D$ 中的所有点的平方误差 $E_i$，并与前一次的误差 $E_{i\text{-}1}$ 比较。

$E_i=\sum_{i=1}^{k}\sum_{t=1}^{n_i}\left|P_{it}-\mu_i^{j+1}\right|$，若 $\left|E_{i+1}-E_i\right|<\delta$ 则算法结束，否则转入(2)进行再一次迭代。

初始聚类中心的选取采用最大最小距离选心法。该方法的原则是使各初始类别之间尽可能地保持足够远的距离。首先在用户数据中按等间距抽取一个数据集合：

$X=X_1,X_2,...,X_n$ 式中：$n$为抽样个数，然后按以下步骤进行处理：

（1）取抽样集合中任一用户作为第一个初始聚类中心。记为 $X_1$；

（2）计算$X_1$与其他各抽样点之间的距离 $D_i$，取与之距离最远的那个抽样点作为第二个初始聚类中心，记为 $X_2$。

（3）对于剩余的各个抽样点，计算它到已有各个初始聚类中心的距离，并取其中最小

的距离作为该点的特征距离。在此基础上，再对所有各个剩余点的特征进行相互比较，取其中最大者，并选择与该最大的特征距离相对应的抽样点作为一个新的初始聚类中心。

（4）重复第（3）步，直到初始聚类中心的个数达到要求为止。

最大最小距离选心法选取的初始聚类中心可以使计算效率最优。

## 5.2 问题一求解

### 5.2.1 注意力机制和 Transformer

注意力（Attention）机制[3]由 Bengio 团队于 2014 年提出并在近年广泛的应用在深度学习中的各个领域，例如在 NLP 中用于定位关键 token 或者特征。注意力机制可以很好地学习序列信息，所以引入注意力机制非常契合本问题。

谷歌团队在 2017 年提出了 Transformer[4]。Transformer 抛弃了传统的 CNN 和 RNN，整个网络结构完全是由 Attention 机制组成。更准确地讲，Transformer 有且仅由 self-Attention 和 Feed Forward Neural Network 组成。一个基于 Transformer 的可训练的神经网络可以通过堆叠 Transformer 的形式进行搭建。Transformer 使用了 Attention 机制，将序列中的任意两个位置之间的距离是缩小为一个常量；其次它不是类似 RNN 的顺序结构，因此具有更好的并行性，符合现有的 GPU 框架。

Transformer 采用注意力机制作为模型的特征处理器，和大多数 seq2seq 模型一样，Transformer 的结构也分为编码器和解码器两个部分，编码器由多个编码块叠加而成，同理解码器部分也类似。同时每个编码块都有两个子层，分别是自注意力层和前馈神经网络层。和编码器部分差不多，其解码器每一个网络块中都多了多头注意力层。为了有效进行参数优化，整个网络使用了残差连接和归一化（Add&Norm），结构如下图所示。

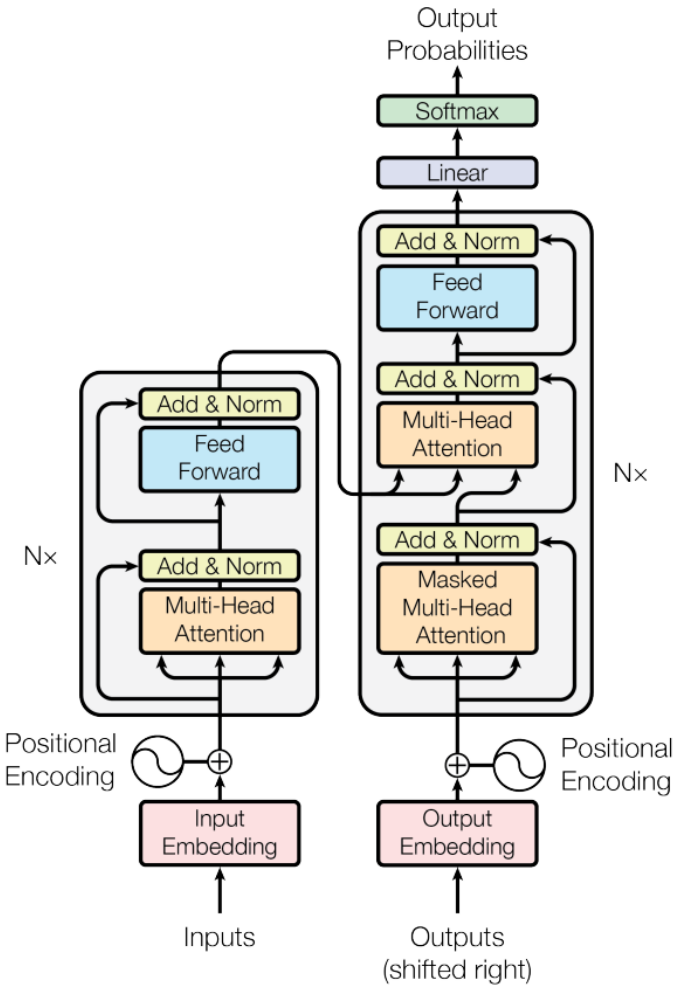



图 11：Transformer 模型架构

*Encoder*

在 Transformer 的 Encoder 中，数据首先会经过自注意力层，Attention 机制的输入包括 3 部分：查询语句 (query)、知识索引 (key) 和知识内容(value)，符号表示分别为：$\mathrm{Q} \in R^{N*d_{emb}}$、$\mathrm{K} \in R^{M*d_{emb}}$ 和 $\mathrm{V} \in R^{M*d_{emb}}$，其中 N 表示 query 的句子长度，M 表示知识索引 key 和知识内容 value 的数目。

query 利用 Attention 机制从 value 中获取信息，key 代表 value 与 query 进行交互，从而决定哪部分知识信息被 query 提取。值得注意的是 key 与 value 的长度都是 M，代表它们之间信息槽位是一一对应的。query 从 value 中提取信息的方法具体如下，

$$\mathrm{info} = \mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)\mathrm{V}$$

$info_i \in R^d$ 表示 query 的第 i 个推荐内容从 value 中抽取的相关信息。首先计算 Q 与 K 的相关矩阵，相关矩阵描述了 value 中的每个信息与 query 的相关度，通过 Softmax 将相

关矩阵在 key 所在的维度归一化，然后通过矩阵乘法计算 Q 从 V 中抽取的信息。Feed Forward 组件由线性网络和激活函数组成，计算公式如下，

$$\text{FeedForward}(\text{X}) = W_2 \max(0, W_1 X + b_1) + b_2$$

其中$\text{X} \in R^{N*d_{emb}}$的表示序列的编码矩阵，N 为句子长度。

***Decoder***

在解码器中，Transformer block 比编码器中多了个 Encoder-Decoder Attention。在 Encoder-Decoder Attention 中，Q 来自于解码器的上一个输出，K 和 V 则来自于编码器的输出。其计算方式和 Self-Attention 完全相同。

***Multi − Head Attention***

Multi-Head Attention 是在 Attention 的基础上添加了多次并行相关度计算。Multi-Head Attention 机制的计算过程如下所示，

$$head_i = Attention\left(QW_i^Q, KW_i^K, VW_i^V\right)$$

$$\text{MultiHead}(\text{Q}, \text{K}, \text{V}) = \text{Concat}(head_1, \dots, head_h)W^O$$

首先将 Q,K 和 V 分别通过 h 次不同的线性变换（线性映射矩阵为$W_i^Q$、$W_i^K$、$W_i^V$），再将 h 次映射结果分别通过 Attention 机制计算信息提取结果$head_i$，最后将得到的多个信息提取结果$head_{i\,i=1}^{\,h}$拼接并进行线性变换得到最终的信息向量。Multi-Head Attention 共进行了 h 次相关度计算，每次相关度计算从 value 中提取出不同的知识信息，进而丰富信息内容。

**残差连接和层归一化**

编码器采用了残差网络中的 short-cut 结构，即每个编码器中的每个子层(self-attention, ffnn)周围都有一个残差连接，然后是一个层归一化步骤。后续通过 Feed Forward 层、残差机制和 Layer Normalization 得到最终的输出结果$X_2$，

$$X_2 = LayerNorm(X + X_1)$$

### 5.2.2 基于 Transformer 的深度学习模型

基于$Transformer$的深度学习模型，首先进行特征提取，$Encoder$的是用户和时间，每个 6 维，一共 12 维嵌入用户信息和推荐内容。[5]$Decoder$输入是推荐内容（$item$），输出是每条内容的浏览时间$Duration$。

模型的出发点是用户的浏览体验，考虑人的精力是有限的，看到喜欢的内容，如

果花费较长的时间浏览，势必会消耗大量的精力，但对于系统的推荐仍是满意的。如果遇到不喜欢的推荐内容，就会影响浏览心情，从而不满足推荐的内容，也会很大程度的影响后序内容的浏览。基于上述观点我们可以得到浏览时间$duration$的以下四种解释：

1. $D_t > 0$且$|D_t|$很大，代表用户花了很长时间在看喜欢的内容，浏览了多少时间，精力就下降了多少。

2. $D_t > 0$ 且$|D_t|$不大，代表用户点击并匆匆浏览了自己喜欢的内容。

3. $D_t < 0$ 且$|D_t|$很大，代表用户讨厌这个内容，看完会感觉不适，结束了浏览。

4. $D_t < 0$ 且$|D_t|$很小，代表用户没有点击内容，直奔这下一条去了。

当浏览完$t$条推荐内容后，得到的$D_t > 0$,代表用户点击该条推荐内容，并且$D_t$就是浏览时长。如果$D_t = 0$代表用户没有点击该条推荐内容，浏览时长为 0。

对一个序列按顺序进行计算，直至在计算某一物品时得到的 Energy < Terminal_Energy，解释为用户看完了，此时对浏览时长和点击量求和,已知标签: $D \in R^N \times R^T$，网络输出: $P \in R^N \times R^T$，总能量$E \in R$设$H(x) = 1_{x>0}$基于浏览时间和点击数目构造**损失函数(loss function)**：

$$\sum_{i=1}^{N}\sum_{j=1}^{T}\mathbf{1}_{d_{ij}>0}\cdot\left(D_{ij}-E\cdot P_{ij}\right)+\sum_{i=1}^{N}\sum_{j=1}^{T}\mathbf{1}_{d_{ij}=0}\cdot sign\left(P_{ij}\right)+\left(1-\sum_{i=1}^{N}\sum_{j=1}^{T}\left|\mathbf{1}_{d_{ij}>1}\cdot P_{ij}\right|\right)^2$$

- 标签是用户看每个$item$的时间长度，如果标签为正，也损失为差值平方；如果标签为负，则为输出的符号函数；最后再加一项剩余能量。第一项表示对于标签为正的元素，希望其越接近网络输出越好，解释为用户浏览至该内容后，在该内容上花费的时间；
- 第二项表示若网络输出零的元素，希望其网络输出为负数，代表其消耗了用户的能量；
- 第三项表示用户在结束浏览时的能量越接近零越好，表示用户不论是点击并浏览内容，还是放弃浏览，均消耗一定的能量（点击并浏览时消耗的能量数值上等同于浏览时间；而放弃浏览时耗费的能量可以理解为用户对该内容的厌恶程度），直至能量为负时结束浏览。

### 5.2.3 基于最相似用户数据的估计模型

1. 选取最相似的用户

对于训练集中出现的用户, 其最相似的用户即为本身.对于训练集中未出现的用户, 统计其被推荐内容所在分类并归一化, 得到$x \in R^{1485}$.对于每个忠实用户也相应地统计其被推荐内容所在分类并归一化, 得到$A \in R^{218744} \times R^{1485}$.将最相似的用户定义为$argmax(Ax)$所对应的用户. 举例如下:

对于测试集中 ID 为 1439416582 的用户和其在训练集中最相似的 ID 为 1376133776 的用户, 其被推荐内容所在类别的频率直方图分别如下所示:

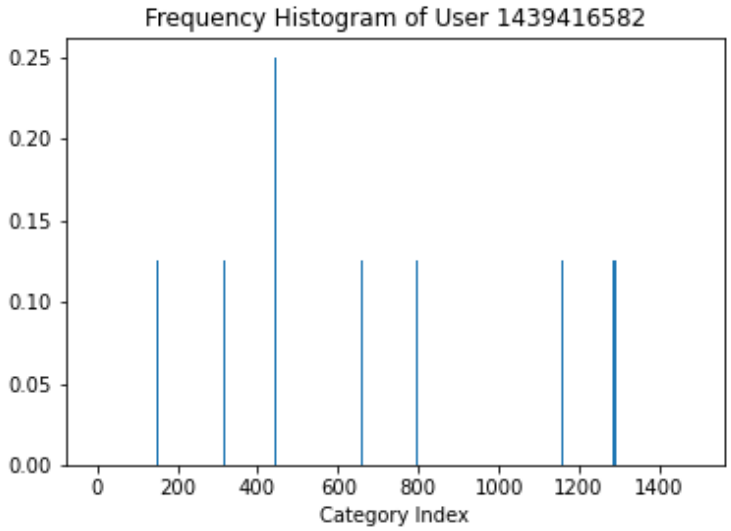



图 12： ID 为 1439416582 的用户被推荐内容所在类别的频率直方图

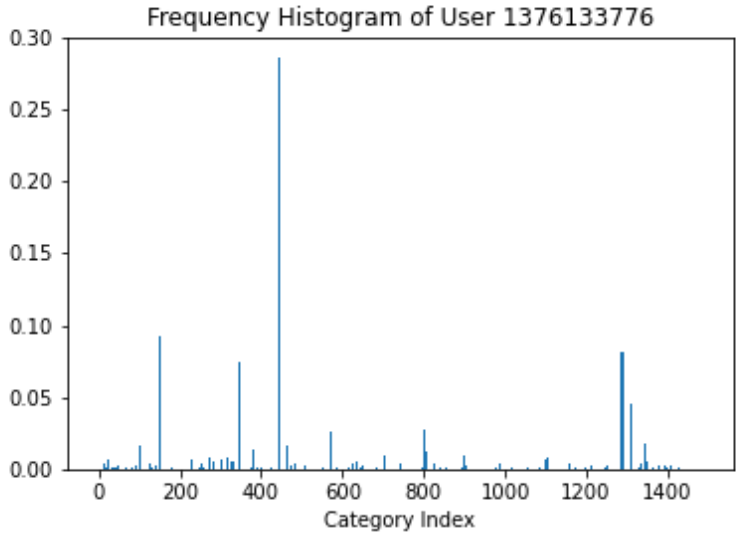



图 13：ID 为 1376133776 的用户被推荐内容所在类别的频率直方图

2. 预测点击率和浏览时长

计算上述得到最相似用户的$Clickeds \in R^{1485}$和Durations $\in R^{1485}$, 则$x \cdot Clickeds$为预测点击率, $x \cdot Clickeds \odot Durations$为预测浏览时长. 下展示部分预测结果:

| index | clicked | duration |
|---|---|---|
| 0 | 1.037865 | 347.127631 |
| 1 | 0.544133 | 116.417053 |
| 2 | 0.953476 | 181.991922 |
| 3 | 1.893914 | 151.787468 |
| 4 | 0.441185 | 31.864015 |
| ... | ... | ... |
| 99995 | 0.533333 | 94.466667 |
| 99996 | 0.913690 | 165.532738 |
| 99997 | 1.529221 | 259.574026 |
| 99998 | 0.720779 | 150.935065 |
| 99999 | 0.282243 | 57.512495 |

图 14：基于相似用户算法的部分预测结果

### 5.2.4 基于 Transformer 和相似用户估计的混合模型

混合模型结合了 5.2.2 基于 Transformer 的深度学习模型和 5.2.3 基于相似用户数据的估计方法。对于两种方法求出来的值简单地取加权求和,

$$N^i_{Clickeds} = \alpha NA^i_{Clickeds} + \beta NB^i_{Clickeds}$$

$$N^i_{\mathrm{Durations}} = \alpha NA^i_{\mathrm{Durations}} + \beta NB^i_{\mathrm{Durations}}$$

$$\alpha = \beta = 0.5$$

$N^i_{Clickeds}$是测试集中第 i 条推荐内容用户的总点击量，$NA^i_{Clickeds}$ 是基于深度神经网络模型预测的第 i 条推荐内容用户的总点击量，$NB^i_{Clickeds}$基于相似用户估计预测的第 i 条推荐内容用户的总点击量。$NB^i_{Clickeds}$是基于相似用户估计预测的第 i 条推荐内容用户的总点击量，$NA^i_{\mathrm{Durations}}$是基于深度神经网络模型预测的第 i 条推荐内容用户总浏览时长，$NB^i_{\mathrm{Durations}}$是基于相似用户估计预测的第 i 条推荐内容用户的总浏览时长。

由于官方没有给出评估准则, 因此无法判断两个模型孰优孰劣, 取平均结合了 transformer 提取序列信息和时间信息的优点, 以及传统统计方法的可解释性，预计会有更好的效果。

由于基于相似用户估计的模型无法获得内容的序列信息，我们根据 Transformer 模型的输出得到如下基于点击收益和时长收益的综合收益评估模型，

$$\frac{\sum_{j=1}^{T} NA^i_{Clickeds} - \mu_{Clickeds}}{\sigma_{Clickeds}} + \frac{\sum_{j=1}^{T} NA^i_{Durations} - \mu_{Durations}}{\sigma_{Durations}}$$

其中$\mu_C$和$\mu_D$分别代表分别表示训练集中一条请求的点击数和总浏览时长的均值，$\sigma_D$和$\sigma_D$分别表示训练集中一条请求的点击数和总浏览时长的标准差。

## 5.3 问题二求解

根据问题一训练得到网络模型就可以预测测试集中推荐序列的总点击量和总时长。

具体结果放到附加文件 result.csv 中提交，这里展示部分结果，第一列是请求 ID、第二列是总点击量、第三列是总浏览时长。

| | A | B | C |
|---|---|---|---|
| 1 | 719708291_1635480753679_2960 | 1.037865 | 347.1276306 |
| 2 | 679978594_1635491281923_1780 | 0.544133 | 116.417053 |
| 3 | 781051348_1635478916952_3923 | 0.953476 | 181.9919215 |
| 4 | 1223262997_1635514428746_6191 | 1.893914 | 151.7874682 |
| 5 | 1210159908_1635487197871_9372 | 0.441185 | 31.86401494 |
| 6 | 1206224433_1635467439702_8607 | 0.2 | 0.2 |
| 7 | 1202318116_1635482087449_3598 | 0.703704 | 42.74074074 |
| 8 | 1209354488_1635501697466_3893 | 0.337035 | 17.03266199 |
| 9 | 1185220039_1635499119996_2020 | 0.346149 | 33.30818971 |
| 10 | 1104284481_1635502336773_3318 | 0.16129 | 42.61290323 |
| 11 | 1113175175_1635481859222_1855 | 0 | 0 |
| 12 | 923868160_1635492811503_5662 | 0.048357 | 3.938200149 |
| 13 | 652686967_1635459880177_6714 | 0.255556 | 112.6666667 |
| 14 | 1209895755_1635484546534_7699 | 0.882448 | 176.9665862 |
| 15 | 694715900_1635499307518_3951 | 0.913978 | 155.1792115 |
| 16 | 1213591859_1635470224158_1276 | 1.016377 | 132.8631066 |
| 17 | 1110057206_1635467007878_1365 | 1.635369 | 450.5437788 |
| 18 | 1105606667_1635513224577_6747 | 1.194996 | 130.6857971 |
| 19 | 710559822_1635483096347_7092 | 0.861111 | 265.4722222 |
| 20 | 1097157083_1635513341928_5069 | 0.392857 | 37.07142857 |
| 21 | 281697980_1635481807547_4247 | 0.448227 | 49.27234043 |
| 22 | 555753684_1635484583256_7842 | 0.246667 | 57.16047619 |
| 23 | 1112874791_1635499702797_4116 | 1.5 | 224.3333333 |
| 24 | 1225603953_1635477456609_813 | 0.208676 | 7.365537381 |

图 15：部分预测结果

## 5.4 问题三求解

### 5.4.1 集束搜索

集束搜索（Beam Search），又称为定向搜索，是一种启发式的图搜索算法。[6]集束搜索算法在每一步深度扩展的时候，在解空间比较大的情况下，为了减少搜索所占用的空间和时间，会剪掉一些质量比较差的结点，保留下一些质量较高的结点。这样能够减少了空间消耗，并提高时间效率，是一种最佳优先搜索算法的优化。在最佳优先搜索算法中，对所有可能的解依据启发式规则进行排序，以衡量得到的解与目标解到底有多接近。而集束搜索，使用宽度优先搜索来构建它的搜索树，在每个深度中裁减掉的其他节点，仅保留 m 个最符合条件的解节点，并使用启发式函数评估它检查的每个节点的能力。其算法的步骤为：1、将初始节点插入到 list 中；2、将给节点出堆，

如果该节点是目标节点，则算法结束；3、否则扩展该节点，取集束宽度的节点入堆。然后到第二步继续循环。4、算法结束的条件是找到最优解或者堆为空。

启发式搜索利用问题拥有的启发信息来引导搜索，达到减少搜索范围、降低问题复杂度的目的。集束搜索可以认为是维特比算法的贪心形式，贪心算法是先挑出一个最可能的词，在此基础上再挑出第二个最有可能的词，以此类推，使每个词的概率最大化，在维特比搜索中由于利用动态规划导致当字典较大时效率低，而集束搜索使用 beam size 参数来限制在每一步保留下来的可能性词的数量，站在整个词序列整体的角度上使其概率最大化。

在本问题中，设置参数集束宽度（beam width），表示在生成每个解码结果时会考虑集束宽度个候选结果。例如假设集束宽为 3，在生成结果的第一个内容时，要求的概率为 $P(y^{<1>} \mid x)$，x 为输入的内容序列。假设经过学习后 $y^{<1>}$ 的 3 个候选为：a, b, c。然后第二个内容在确定第一个内容的基础上继续搜索，此时第一个和第二个内容的联合概率是关心的重点，即：

$$P(y^{<1>}, y^{<2>} \mid x) = P(y^{<1>} \mid x) P(y^{<2>} \mid x, y^{<1>})$$

由于此处集束宽为 3，所以 3 个候选内容 a,b,c 都要重复上述操作。假设字典中内容数为 n，那么考虑第二步会产生 3n 个可能的结果，通过计算得到所有组合的联合概率 $P(y^{<1>}, y^{<2>} \mid x)$,计算后再从 3n 个结果中挑选出联合概率最高的 3 个结果作为候选存于内存，同时保存每个候选的 $P(y^{<1>}, y^{<2>} \mid x)$,以便最终计算整个序列的概率。通过计算留下概率最大的三个候选序列，并以此类推，得到整个推荐序列的收益结果。

集束搜索可以运用于强化学习的解码策略中，以一定时空复杂度为代价，提升解码所得解的性能。

### 5.4.2 强化学习

使用强化学习的方法解决从 N 个候选内容中选取 K 个候选内容，选取的标准是，希望用户的总点击量尽可能多和总浏览时长尽可能的长。基于上述两个目标构造回报函数（reward function）：

$$R = \frac{\sum_{j=1}^{T} C_j - \mu_C}{\sigma_C} + \frac{\sum_{j=1}^{T} D_j - \mu_D}{\sigma_D}$$

其中$\mu_C$和$\mu_D$分别表示训练集中一条请求的点击数和总浏览时长的均值，$\sigma_C$和$\sigma_D$分

别表示训练集中一条请求的点击数和总浏览时长的标准差。

结合 transformer 模型进行求解，transformer 模型的$Encoder$和$Decoder$部分按照下面的公式设计。它是一个标准的 Transformer 编码器，具有多头注意力和残差连接。 唯一的区别是使用批量归一化，而不是层归一化。 内存/速度复杂度为 $O(N^2)$。

$Encoder$:

$$H^{enc} = H^{\ell} = L^{enc} \in R^{N\times d},$$

其中,

$$H^{\ell} = 0 = Concat(z, X) \in R^{N\times 2}, z \in R^{2}, X \in R^{n\times 2},$$

$$\mathrm{H}^{\ell+1} = \mathrm{softmax}\left(\frac{\mathrm{Q}^{\ell}\mathrm{K}^{\ell^{\mathrm{T}}}}{\sqrt{(\mathrm{d})}}\right)\mathrm{V}^{\ell} \in \mathrm{R}^{N\times \mathrm{d}},$$

$$Q^{\ell} = H^{\ell}W_Q^{\ell} \in R^{N\times d}, W_Q^{\ell} \in R^{d\times d},$$

$$K^{\ell} = H^{\ell}W_K^{\ell} \in R^{N\times d}, W_K^{\ell} \in R^{d\times d},$$

$$\mathrm{V}^{\ell} = \mathrm{H}^{\ell}\,\mathrm{W}_{\mathrm{V}}^{\ell} \in \mathrm{R}^{\mathrm{N}\times \mathrm{d}}, \mathrm{W}_{\mathrm{V}}^{\ell} \in \mathrm{R}^{\mathrm{d}\times \mathrm{d}},$$

解码是自回归的，一次一个推荐内容。 假设我们已经解码了推荐序列中的前 t 条推荐内容，并且我们想要预测下一条内容。 解码过程由下面详述的四个步骤组成。

$Decoder$:

1.从之前选择的$i_t$推荐内容的编码开始解码

$$H_t^{dec} = h_{i_t}^{enc} + PE_t \in R^{d},$$

$$H_t^{dec} = 0 = h_{start}^{dec} = z + PE_t = 0 \in R^{d},$$

$$PE_{t,i} = \begin{cases} \sin(2\pi \mathrm{f_i t})\mathrm{i} \text{ 是奇数}, \\ \cos(2\pi \mathrm{f_i t})\mathrm{i} \text{ 是偶数}, \end{cases} \quad f_i = \frac{10{,}000^{\left|\frac{d}{2i}\right|}}{2\pi}$$

2.对部分推荐序列使用自注意力查询。 自注意力层使用多头注意力、残差连接和层归一化。在解码步骤 $t$ 处，内存/速度复杂度为 $O(t)$。 具体公式如下：

$$\hat{h}^{\ell+1} = softmax\left(\frac{q^{\ell}K^{\ell^{T}}}{\sqrt{(d)}}\right)V^{\ell} \in R^{d}, \ell = 0, \ldots, L^{dec} - 1$$

$$\mathrm{q}^{\ell} = \hat{\mathrm{h}}_{\mathrm{t}}^{\ell}\widehat{\mathrm{W}}_{\mathrm{q}}^{\ell} \in \mathrm{R}^{\mathrm{d}}, \widehat{\mathrm{W}}_{\mathrm{q}}^{\ell} \in \mathrm{R}^{\mathrm{d}\times \mathrm{d}},$$

$$K^{\ell} = \widehat{H}_{1,t}^{\ell}, \widehat{W}_K^{\ell} \in R^{t\times d}, \widehat{W}_K^{\ell} \in R^{d\times d},$$

$$\ell = \widehat{H}_{1,t}^{\ell}, \widehat{W}_V^{\ell} \in R^{t\times d}, \widehat{W}_V^{\ell} \in R^{d\times d},$$

$$\widehat{H}_{1,t}^{\ell} = \left[\hat{h}_1^{\ell}, .., \hat{h}_t^{\ell}\right], \hat{h}_t^{\ell} = \begin{cases} h_{tec}^{d}, if\ell = 0 \\ h_t^{q,\ell}, if\ell > 0 \end{cases}$$

3.使用查询注意力层查询未选中的推荐内容中的下一条可能的推荐内容。 使用了多头

注意力、残差连接和层归一化。每个递归步骤的内存/速度复杂度为 $O(K)$。

$$h_t^{q,\ell+1} = softmax\left(\frac{q^\ell K^{\ell^T}}{\sqrt{(d)}} \odot M_t\right) V^\ell \in R^d, \ell = 0, \dots, L^{dec} - 1$$

$$q^\ell = \hat{\mathrm{h}}_{\mathrm{t}}^{\ell+1} \widetilde{W}_{\mathrm{q}}^{\ell} \in R^d, \widetilde{W}_{\mathrm{q}}^{\ell} \in \mathrm{R}^{\mathrm{d}\times\mathrm{d}},$$

$$K^\ell = H^{enc} \widetilde{W}_K^\ell \in R^{t\times d}, \widetilde{W}_K^\ell \in \mathrm{R}^{\mathrm{d}\times\mathrm{d}},$$

$$V^\ell = H^{enc} \widetilde{W}_V^\ell \in R^{t\times d}, \widetilde{W}_V^\ell \in \mathrm{R}^{\mathrm{d}\times\mathrm{d}},$$

⊙是 Hadamard 积。

4.使用单头注意力执行最终查询以获取未选取推荐内容分布的最后一步。 最终，在训练期间使用伯努利从分布中采样下一个节点$i_{t+1}$，并在推理时使用贪心（具有最大概率的索引）来评估基线。 内存/速度复杂度为 $O(K)$。

$$p_t^{dec} = softmax\left(C.tanh\left(\frac{qK^T}{\sqrt{(d)}} \odot M_t\right)\right) \in R^n,$$

$$q = h_t^q \overline{W}_q \in R^d, \overline{W}_q \in R^{d\times d},$$

$$K = H_K^{enc} \overline{W} \in R^{n\times d}, \overline{W}_K^\ell \in R^{d\times d},$$

$$C = 10$$

剪枝选取长度为$K$的推荐序列，可以转化为序列优化问题，

$$max_{seq_K=\{i_1,\dots,i_K\}} P^{item}(seq_K|X) = P^{item}(i_1, \dots, i_K|X).$$

对于自回归解码，即一次选择一个推荐内容，$P^{item}$可以用链式法则分解：

$$P^{item}(i_1, \dots, i_K|X) = P(i_1|X) \cdot P(i_2|i_1, X) \cdot \dots \cdot P(i_K|i_{K-1}, i_{K-2}, \dots, X).$$

因此，解码问题旨在找到序列$i_1, \dots, i_K$,其中最大化目标：

$$max_{i_1,\dots,i_K} \prod_{t=1}^{K} P(i_t|i_{t-1}, i_{t-2}, \dots i_1, X)$$

复杂度是 $O(K!)$，通过穷举搜索准确找到最佳序列是很困难的，所以需要近似搜索。最简单的近似搜索是贪心搜索；在每个时间步，以最高概率选择下一个推荐内容：

$$i_t = argmax_i P(i|i_{t-1}, i_{t-2}, \dots i_1, X)$$

复杂度是线性的 $O(K)$。

集束搜索是一种广度优先搜索 $(BFS)$ 技术，是弱化版的贪心策略，其中宽度的大小 $B$ 有限。采用集束搜索结合强化学习的方法[7]，集束搜索解码问题变成以下形式：

$$max_{\{i_1^b,\dots,i_K^b\}_b^B} \prod_{b=1}^{B} P\left(i_1^b, \dots, i_n^b \middle| X\right) s.t. \{i_1^b, \dots, i_n^b\} \neq \{i_1^{b'}, \dots, i_n^{b'}\}, \forall b \neq b'$$

对于 $B = 1$，通过贪心解码给出解决方案。 对于$B > 1$，通过考虑 $B$集束的所有可能扩展来确定 $t$ 处的解，并且仅保持 $Top - B$ 概率：

$$\{i_1^b, \dots, i_K^b\}_b^B = \mathrm{Top-B}\{\prod_{k=1}^{t} P\left(i_k^b | i_{k-1}^b, \dots, i_1^b, X\right)\}_{\mathrm{b=1}}^{\mathrm{B(n-t)}},$$

我们提出的解决方法就是集束搜索结合$transformer$的强化学习算法。

# 六．模型分析

## 6.1 优缺点分析

### 6.1.1 模型的优点

1. Transformer 是序列到序列模型的经典方法,训练效率高，可并行化,并且充分考虑了内容排列组合对收益大小的影响，对不同顺序的推荐序列的偏好非常敏感。
2. 在建模过程中添加了能量这一用户属性，使得模型可以预测用户与每个被推荐内容的交互情况，而不是整个推荐序列的整体点击量和浏览总时长。这一步骤使得模型有充分的可解释性，在解码时能够模拟用户真实的阅读场景，能够分辨出用户厌恶哪些内容，完美地捕捉用户的偏好，精准地给用户推送使得平台收益最高的内容。
3.在深度学习模型的基础上结合传统的统计方法进行预测，不仅可以比较两者的预测结果从而评估模型的训练程度，也可以将两者结合，汲取 Transformer 擅于捕捉序列特征的优点及传统方法的可解释性，大大提高模型的预测精度。

### 6.1.2 模型的缺点

1. transformer 模型需要很大的算力，去分析每个用户的浏览习惯和内容偏好。

# 七．参考文献

[1]林俊俊.面向用户个性化兴趣准确表达的推荐算法研究[D].电子科技大学, 2021. DOI:10.27005/d.cnki.gdzku.2021.002087.
[2]王伟. Kmeans 聚类与多波谱阈值相结合的云检测和烟检测算法研究[D].中国科学技术大学,2011.
[3] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate[J]. arXiv preprint arXiv:1409.0473, 2014.
[4] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need [C]//Advances in Neural Information Processing Systems. 2017: 5998-6008.
[5] Kazemi, Seyed Mehran, et al. "Time2vec: Learning a vector representation of time." arXiv preprint arXiv:1907.05321 (2019).
[6]史宇. 基于深度学习的中文分词方法研究[D].南京邮电大学, 2019. DOI:10.27251/ d.cnkl.gnjdc.2019.001283.
[7] Bresson, Xavier, and Thomas Laurent. "The Transformer Network for the Traveling Salesman Problem." arXiv preprint arXiv:2103.03012 (2021).